COMP4436

Subject Title: ARTIFICIAL INTELLIGENCE OF THINGS

(Dr. Aquil Mirza)

Group 12

Group Assignment I

Comparative Analysis of ML, DL and SNN Algorithms

Total: 8 page

Group member:

Yeung Hang (22027226d)

Yeung Tsz Kwan (23103029d)

Chan Kin Wang (23031551d)

The Enhanced Performance of Spiking Neural Networks

SNN may prove to be a perfect substitute for several Artificial Intelligence on the Internet of Things (AIoT) applications. For image classification tasks, there are clear benefits to using convolutional neural networks (CNNs) in traditional deep learning. Even, spiking neural networks (SNNs) can be utilized instead of more conventional deep learning architectures. In this report, we will comparative analysis and focus on how SNNs compare with four other algorithml including of two machine learning, two deep learning. Both will be having one of supervised and unsupervised learning. We will also include charts to help visualize the results. Our findings demonstrate that better classification performance can be obtained by integrating CNN feature extraction into the SNN framework. This clarifies that SNN can extract particular features for SNN model training using a variety of approaches in various contexts.

The rapid development of Artificial Intelligence on the Internet of Things (AIoT) is the result of incorporating artificial intelligence algorithms into IoT devices to achieve intelligent decision-making at the edge. Such as in AIoT applications such as smart home, healthcare, and surveillance. Image analysis is a crucial task because of its scope. It can not only be used for object detection but also in medical imaging, autonomous driving, security monitoring and other AI operations that will be realized in the future. Image classification is a basic task. In order to understand whether there are other suitable alternatives to CNN in image classification, this article will classify photos of cats and dogs and compare five technologies in detail: Spiking Neural Network (SNN), Convolutional Neural Network (CNN), Autoencoder, K-means clustering and logistic regression. The results show that SNNs can be comparable to other traditional machine learning (ML) and deep learning (DL) methods in terms of accuracy, precision, recall, and F1 score. Among them, SNN is combined with CNN for feature extraction and SNN for classification. This demonstrates the potential advantages of SNNs in AIoT applications, where extracting specific features in various environments enhances their energy efficiency, real-time processing, and noise resistance. This enables SNN to be used as an alternative for different Artificial Intelligence of Things (AIoT) applications.

---

1. Performance evaluation indicators

This report importing four important indicators were introduced in the study, each of which corresponds to the actual results of the algorithm. These metrics include accuracy, precision, recall, and F1-score, and these performance evaluation metrics are as follows:

1.1 Accuracy: The ratio of correctly predicted instances to the total

$$\text{Accuracy} = \frac{True\ Positives + True\ Negatives}{Total\ Instances}$$

1.2 Precision: The ratio of correctly predicted positive observations to the total predicted positives.

$$\text{Precision} = \frac{True\ Positives}{True\ Positives + False\ Positives}$$

1.3 Recall: The ratio of correctly predicted instances to the total

$$\text{Recall} = \frac{True\ Positives}{True\ Positives + False\ Negatives}$$

1.4 F1-score: The ratio of correctly predicted instances to the total

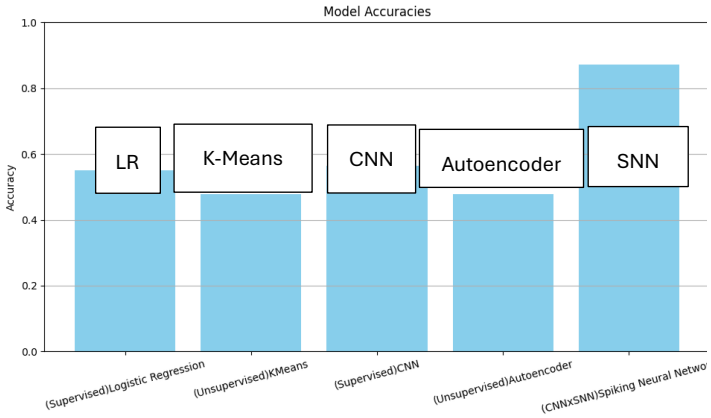
$$\text{F1} - \text{score} = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Metric	Formula	Best Use Case
Accuracy	(TP+TN)/(TP+TN+FP+FN)	Balanced classes
Precision	TP/(TP+FP)	High cost of false positives
Recall	TP/(TP+FN)	High cost of false negatives
F1-Score	$2 \times \frac{Precision \times Recall}{Precision + Recall}$	Imbalanced classes

2. Dataset Experiemental Setup

The CatsDogsDataset is utilize a dataset containing images of cats and dogs from (https://www.kaggle.com/datasets/samuelcortinhas/cats-and-dogs-image-classification). Which dataset is divided into training set and validation set. The dog and cat images of the dataset will include different appearances, styles, quantities, illustrations and hand drawings. Total contains 279 cat images and 278 dog images as the training set and contains 70 cat images and 70 dog images as the validation set.

Five Model's Performance of Accuracy (Fig. 1.)



Logistic Regression: 0.550	K-Means: 0.478571
Autoencoder: 0.478571	CNN: 0.564286
SNN x CNN: 0.870736	

1. SNN Algorithum

1.1 Advantages of SNN in AIoT and Performance Analysis

The results of Fig. 1., and Table 1 show that the CNN apply on the SNN can be comparable to the Machine Learning (Logistic Regression and K-Means clustering) and Deep Learning (Autoencoder and Convolutional Neural Network) models in terms of accuracy, precision, recall, and F1 score.

By outperforming both traditional CNN and logistic regression models in accuracy, recall, and F1-score, SNN demonstrate their potential as a powerful tool for AIoT applications. As for the higher precision and recall, it means that the SNN can more definitively correctly identify images of cats and dogs. Which is useful with the AIoT security process like cramped environment.

This allows for more precise feature discrimination compared to CNN by extraction with multiple models on SNN. Which SNN-based model can also approach on a lot of different situations and also particularly well suited for AIoT applications that require energy efficient, high-accuracy classification. Such as smart surveillance, autonomous vehicles for real-time object detection

1.2. SNN break through limitations of these Models in AIoT

CNN has performed well in image classification, although there are still limitations in AIoT applications. CNN is a deep learning model used primarily for image processing. They are built with convolutional layers that extract spatial features from images and pooling layers that reduce dimensions, allowing them to automatically learn hierarchical features and finally achieve cutting-edge performance in image classification tasks.

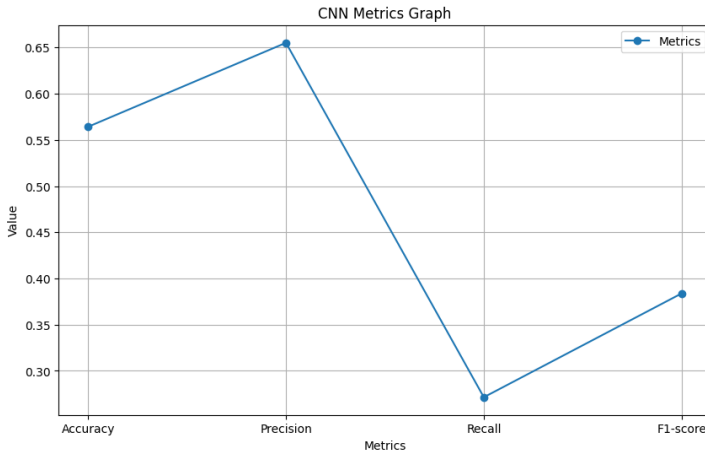
However, noise, distortion, poor contrast and clutter in the background of the dataset or inappropriate preprocessing may cause the CNN training dataset be overfitting with the specific features. This greatly limits the trained model's ability to generalize to unfamiliar input. Within this constraint, SNN can be integrated with several models to prevent overfitting, optimize CNN feature extraction capabilities, and allow SNN to overcome CNN limits.

Most importantly, SNN can leverage the strengths of other models by integrating PCA, Autoencoder, K-Means and Logistic Regression. Such as integrating PCA can reduce the input dimensionality before passing data and integrating K-Means clustering for pregrouping with a similar pattern before the SNN model training, etc. Which can optimize feature extraction and ensure SNN is offered with other model's distinct advantages in AIoT applications.

Tabel 1: [Data taken from "All_algorithm.ipynb" of the result graphs]

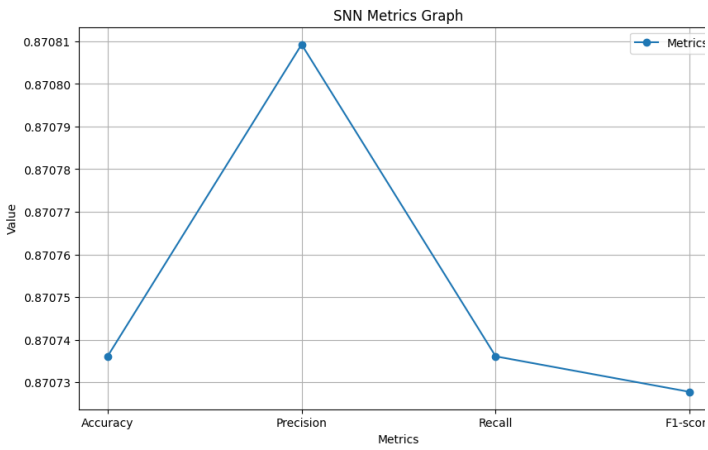
Algorithms	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	Execution Time (sec)
LR	0.550000	0.547945	0.571429	0.559441	0.379000
K-Means	0.478571	0.472727	0.371429	0.416000	0.374604
Autoencoder	0.478571	0.481013	0.542857	0.510067	6.222759
CNN	0.564286	0.655172	0.271429	0.383838	20.804511
SNNxCNN	0.870736	0.870809	0.870736	0.870728	17.175854

CNN Accuracy Graph (Fig. 2.)



Accuracy 0.564286	Precision 0.655172
Recall 0.271429	F1-score 0.383838

SNNxCNN Accuracy Graph (Fig. 3.)



Accuracy 0.870736	Precision 0.870809
Recall 0.870736	F1-score 0.870728

1.3. Feature Extraction with CNN and Its Integration into SNN

Advantages of SNN in Image Classification, one of the key strengths of SNN in image classification, is to serve as a preprocessing layer of CNN to pre-extracted features. The SNN processes raw image input and encodes it into spike-based feature maps. The spike-based features are then fed into a CNN for further spatial analysis. This approach reduces redundant information and enhances edge detection features.

CNNs are well known for their effectiveness in hierarchical feature extraction, capturing spatial patterns in images. By using SNN solely for feature extraction and passing these features into CNN for classification, by extracting high-level spatial features from the images and transforming them into a format that an SNN and CNN can process efficiently. This approach enhances classification accuracy while maintaining computational efficiency, making it particularly suited for edge devices in AIoT.

The results of Fig. 2. and Fig. 3., above illustrates that the SNNxCNN outperforms both CNN in terms of accuracy, precision, recall and F1-Score, with accuracy of SNN's ~ 0.87 compared to CNN's ~0.57, and precision precision of SNN's ~ 0.87 compared to CNN's ~0.66, and recall of SNN's ~ 0.87 compared to CNN's ~0.27, and F1-score of SNN's ~0.87 compared to CNN's ~0.38. Several factors contribute to the superior performance of SNNs with following functions.

1.3.1. Higher Recall with Efficient Feature Utilization

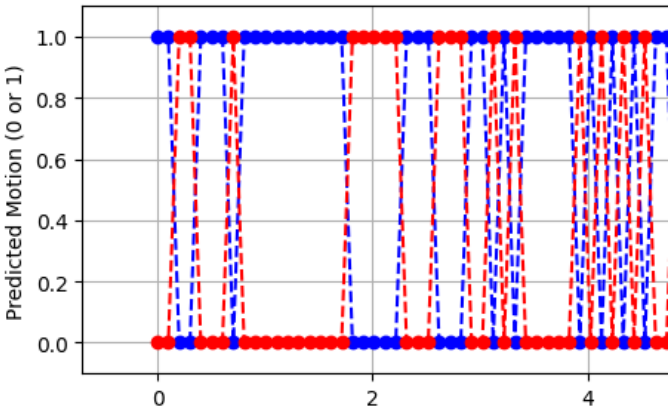
Based the CNN are a class of deep learning models specifically designed for image processing. Which excel at extracting spatial features from images, such as edges, textures, and shapes. By applying the SNN into the CNN model as the feature extraction input and passing these features into which model to used SNN as a feature extractor, the CNN model can leverage these high-level features for classification, improving accuracy and robustness. Also, CNN handles the computationally intensive task of feature extraction, while the SNN performs lightweight classification, making the overall model suitable for deployment on edge devices and making them ideal for AIoT applications. So, the combination of CNN and SNN allows the model to scale efficiently.

However, which testing is not applying other models in the SNN by integrating PCA, Autoencoder, K-Menas, Logistic Regression and only applied on CNN model as the feature extraction. So, SNN can also apply more than one model for transforming raw pixel data into a high-level representation that captures the essential characteristics of the images. Furthermore, if there are too complex input high-dimensional data, SNN can also apply the PCA model to handle the high-dimensional data and map it into low-dimensional space. It can help us discover the most important features in the data and remove the noise in it, thereby simplifying the data and speeding up the training process of machine learning algorithms.

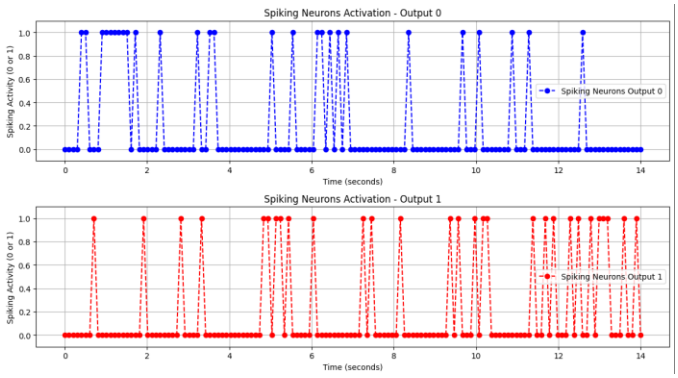
1.3.2. Biologically inspired spiking mechanisms

SNNs are inherently robustness in noisy environments due to their biologically inspired spiking mechanisms. Which is because the extracted features are processed by spiking neurons, which use temporal coding to encode information.

Spiking Neurons Activation Graph (Fig. 4.)



Spiking Neurons Activation Graph between output 0 to 1 (Fig. 5.)



As the spiking neurons only activate when the input reaches a certain threshold, reducing with the enabling processing noise. The results of Fig. 4. and Fig. 5., which display how the spiking neurons are activated by reaching a certain threshold. So, for this combination of CNN and SNN allows the model to achieve high accuracy while maintaining efficiency. In contrast, CNNs often require substantial training and fine-tuning to achieve similar resilience.

SNN superior recall and F1-score indicate that it handles classification tasks with greater robustness, a crucial factor for real-world AIoT applications where sensor noise is common. This is particularly important in AIoT applications, where data quality may be inconsistent due to environmental factors or sensor limitations. So, a lot of AI image processing will also be applied with algorithms to reduce the noise for the hardware to getting the high-quality data image like the rembg algorithm.

2. Machine Learning Algorithm

2.1.1 Experimental Setup - Dataset Preparation

The Experiment setup still used the CatsDogsDataset, while split into training set and validation, the Training set contains 279 cat's images, and 278 dog's images and Validation set contain 70 cats and 70 dog's images, the dog's and cat's images in this dataset include different aspect, spices, amount of them. Even through contain illustrations or hand drawing of them.

2.1.2. Experimental Setup - Pre-processing

The Dataset will be preprocessed by converting the image to grayscale and resizing them to 64x64 pixels and finally flatten to 1D 64X64 = 4096 size of array.

2.1.3. Experimental Setup - Model Usage

The model used in this experiment is Logistic Regression, with maximum of 1000 iterations, The evaluation metrics used to evaluate performance of the model include accuracy, precision, recall and F1-score. The second experiment is using K-Means Clustering instead.

2.1.4. Experimental Setup - Experiment Goal

The Goal of this experiment is obtaining the performance of Logistic Regression on the CatsDogsDataset and compare it with another machine learning models, the K-Means Clustering.

2.1.5. Experimental Setup - Software Requirements

The experiment is conducted using python with NumPy, OpenCV, scikit-learn, pandas and matplotlib libraries. These OpenCV used in image processing, In the Program, After the pre-processing are conducted. The Array will be transmission to Logistic Regression or K-Means model are provided by scikit-learn libraries. The pandas and NumPy are for data processing. And the matplotlib is capable of visualizing the Testing Result.

2.2 Logistic Regression

Logistic regression is a machine learning algorithm used for binary classification tasks. It models the probability of an event occurring by fitting a linear combination of input features to the data using the logistic function, which maps any real-valued number into the range between 0 and 1. The model's parameters are using maximum likelihood estimation, and the coefficients can be explained as the sigmoid of the event occurring. Logistic regression is evaluated using metrics like accuracy, precision, recall and ROC-AUC score.

$$p(x) = \frac{1}{1 + e^{-(x-\mu)/s}}$$

2.3 K-Means Clustering

K-means clustering is a type of unsupervised machine learning algorithm that groups similar data points into clusters based on their features. The algorithm uses an iterative approach to refine the cluster assignments and centroids, resulting in efficient clustering methods. The benefit of K-means clustering is its ability to handle high-dimensional data and large datasets. One of the main challenges is the choice of the number of clusters, which can significantly affect the results.

$$\arg\min_{\mathbf{S}} \sum_{i=1}^{k} \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2 = \arg\min_{\mathbf{S}} \sum_{i=1}^{k} |S_i| \operatorname{Var} S_i$$

2.4. Experiment Procedure

2.4.1. Load the CatsDogsDataset and split it into training and validation set.

2.4.2. Preprocess datasets by converting them to grayscale, resizing the image to 64X64 pixels flat to 4096 size arrays.

2.4.3. Train the Logistic Regression Model on training set with the specified hyperparameters.

2.4.4. Evaluate the performance of the model on the validation set using specified evaluation metrics.

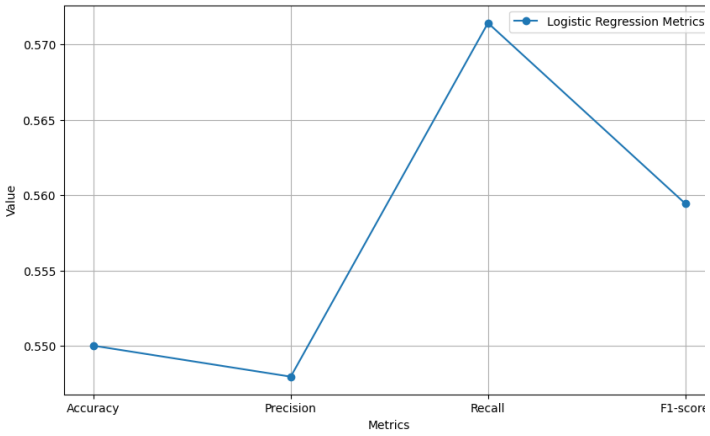
2.4.5. Plot the result using Matplotlib based on the evaluation metrics.

2.4.6. Repeat the experiment with K-Means Clustering and compare their performance among the models.

2.5. Potential limitations

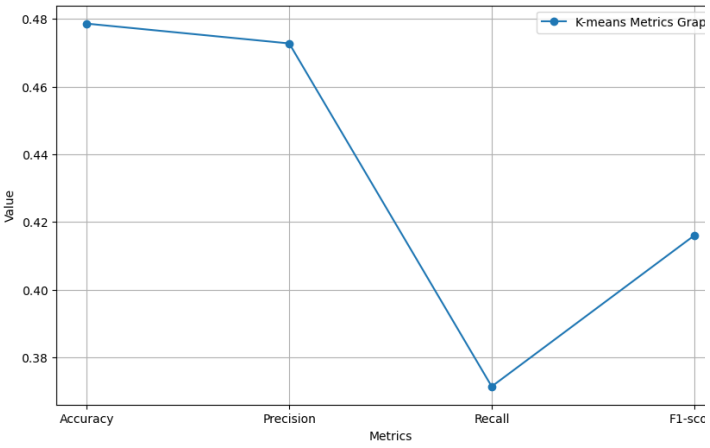
The preprocessing may not be suitable for the Datasets, that may be affected to the model performance. The hyperparameters configuration may not be suitable for the model, that also may impact the model performance.

Logistic Regression Accuracy Graph (Fig. 6.)



Accuracy 0.550000	Precision 0.547945
Recall 0.571429	F1-score 0.559441

K-Means Accuracy Graph (Fig. 7.)



Accuracy 0.478571	Precision 0.472727
Recall 0.371429	F1-score 0.416000

2.6. Result Anaysis

The results of the classification using K-means clustering and logistic regression are shown in the Tabel 1. The logistic regression model reaches an accuracy of 0.55 while K-means clustering achieved an accuracy of 0.479. This indicates that the logistic regression model performance is better than K-means clustering in this dataset in terms of accuracy.

In precision, the logistic regression model has precision of 0.548 and K–means only has 0.472. This suggests that the logistic regression model was model precise in these predictions shown it was more accurate in identifying true positives.

The recall of the logistic regression model was 0.571, while the recall of the K-means clustering model was 0.371. Pointed that the logistic regression model was better detecting true positive, it was more sensitive to the cat dog datasets.

The F1-score of the logistic regression model was 0.560 rather than the K-means clustering 0.416. The F1-score is a measure of the balance between precision and recall, higher score means that have better performance. Therefore, the logistic regression had better performance than K-means clustering in this case.

First, the accuracy of both models is under 0.6, That may indicate the result is not high performance than random guess (0.5), The performance of K-means worse than random guess.

For the low performance. One of the possible results is that the image is a continuing data with all direction pixels, while the flattening of the array will lose the data characteristic.

The K-means have lower performance than random guess due to the cats and dogs they all have similar head and body shape, fur texture and colors, the values of the array may be very similar. That means the distance between the point to point in K-means is very close.
And centroids of the data are very sensitive to the outliners。 That may lead to worse performance of the model.

For the improvement of the result. The convolution technique may be introduced for future experiments, the convolution can maintain the features of the images. A suitable filter can obtain actuated features from images.

3.Deep Learning Algorithm

For this section, the supervised and unsupervised deep learning algorithms chosen are Convoluted Neural Network (CNN) and autoencoder respectively.

3.1 Convolutional Neural Network

CNN is designed specifically to process higher dimensional data, such as images. It functions by extracting features, such as textures and edges, in the image via scanning it with filters. The filters then produce a feature map that highlights the features. It then attempts classification with the produced feature map.

3.1.1 Pre-process

The images are pre-processed by first converting into grayscale 100 x 100 images. They are then applied with Gaussian blur and Sobel filter to reduce noise and highlight edges respectively. Labels are also applied to the images.

3.1.2 Model

The model begins with a convolution layer which applies 32 filters of 3 x 3 size and has the activation function ReLU. The result is then passed into a pooling layer to take the max value of each 2 x 2 window. These layers are then repeated with 64 and 128 filters. The resulting map is then flattened, have values below 0.5 randomly dropped out, passed through a dense layer with 512 neurons and ReLU activation function, and finally classified with softmax function in output. The model is compiled with categorical cross-entropy loss function, Adam optimizer, and accuracy metrics.

3.1.3 Procedure

The model is trained for 20 epochs with processed training images and labels. Also, the training will be stopped early if the validation loss does not improve for 3 consecutive epochs.

3.2 Autoencoder

Autoencoders are composed of “encoders” and “decoders”. They work by having the encoders transform the input into intermediate “encoded” representations with reduced dimensionality. The encoded data is then passed to the decoder, which increases the dimensionality and attempts to reconstruct the input. While some autoencoders can take image input directly, the one that was implemented for this report instead first pre-process the image into reduced vectors and passed into the autoencoder.

3.2.1 Pre-process

The images are pre-processed by converting them into grayscale 64 x 64 images, which then have the values normalized to between 0 and 1, and flattened into vectors of size 4096. The training and testing images are stored separately.

3.2.2 Model construction

The autoencoder is comprised of only the input layer and 1 hidden layer, and the output layer. The hidden encode layer takes in the input and uses the ReLU activation function and produces the reduced output as a size 64 vector. The final decodes layer uses the sigmoid function, and outputs the results as a reconstructed vector of size 4096. The autoencoder is then compiled to use the Adam optimizer and the binary cross-entropy loss function.

3.2.3 Procedure

The model is then trained using the processed training images, which is both the input and the targeted data, meaning the model would attempt to reconstruct the input after deconstructing it. It is then trained for 20 epochs.

3.3 Results analysis

Fig. 8 and 9 show the results of Autoencoders and CNN measured using the metrics mentioned. Autoencoder has an accuracy of 0.478571, precision of 0.481013, recall of 0.542857, and F1-score of 0.510067. Meanwhile, CNN has an accuracy of 0.564286, precision of 0.655172, Recall of 0.271429, and F1-score of 0.383838. It can be seen that CNN has a higher accuracy and precision than autoencoder. On the other hand, autoencoder has a higher recall and F1-score than CNN.

Since CNN has a higher score in both accuracy and precision than autoencoder, it can be said that CNN is better in terms of both correctly classifying and avoiding identifying false positives. However, the higher recall in autoencoder than CNN means that it is better in avoiding identifying false negatives, which has influenced the F1-score of autoencoder to be higher and make it overall better in this regard, despite the lower precision.

The low accuracy and precision of the autoencoder could be possibly due to the simplistic nature of the model, being comprised of only one layer, compared to the 9 layers of the CNN model. This makes it difficult to properly classify the images. However, the same factor could also be possibly the reason it has higher recall. It appears that the autoencoder has identified a lot more positives, whether true or false. Recall is determined only by true positives and false negatives. Since the autoencoder, due to the high number of false positives, has very few of which, this led to its high precision and thus F1-score.

Additionally, both the autoencoder and CNN have high loss during training, which stabilized at about 0.62 and 0.69 respectively, as well as validation loss at around 0.62 and 0.69 respectively, as seen in Fig. 10. While this means both models have stabilized and no overfitting has occurred, it also means both models are not accurate in classification.

Fig. 10 Loss and validation loss at last 3 epochs

Autoencoder		CNN	
Loss	Val. loss	Loss	Val. loss
0.6283	0.6278	0.6987	0.6935
0.6215	0.6261	0.6905	0.6938
0.6220	0.6250	0.6907	0.6935

3.4 Suggestions

In general, both algorithms can potentially improve their performance by pre-processing the images with additional brightness adjustments to mitigate the different lighting in each picture. Fine tuning and repeat experimentation can also be made to the values in the model to achieve the best results.

For the autoencoder, improvements can be made to increase performance by adding more layers to the model to capture the more complex features of the images. Dropout layer and an early stopping function can also be added to reduce overfitting.

For the CNN model, improvements could be made by adding additional batch normalization layers to increase stability and speed, and regularization layer to further reduce overfitting. Additionally, the input could have values normalized to between 0 and 1 to improve the model speed. The effect of the Gaussian blur could also be considered, as noise reduction could also potentially remove finer but crucial details of the images, such as the facial features.